

团队工作流 Team Workflow

1. 协作总则

- 采用 **GitHub Flow**: `main` 是唯一交付主线, 所有改动通过短生命周期分支和 Pull Request 合入。
- 仓库为私有仓库; `protect-main` Ruleset 保护默认分支 `main`。禁止直接推送、删除主干或强推覆盖历史。
- 一人一支、一支一件事; 分支通常存活 1-3 天。优先为行动项建立 Issue, 并在分支名和 PR 中关联编号。
- PR 至少由一位非作者成员 Approve 后才能合入; 平台只允许 **Squash merge**, 合并后删除分支。
- 项目不配置 CI, 本地检查和 PR 中的验证证据是合并前的必要条件。

2. 一项工作的标准闭环

1. **计划**: 在 Issue 或 `process/` 周计划中写明目标、负责人、备份、验收标准和截止时间。
2. **同步主干**: 从最新 `main` 创建分支; 开始提交前确认没有未处理的冲突。
3. **实现与小步提交**: 一次提交只做一件事, 保持改动可审查; 涉及接口变化时先通知消费方。
4. **本地检查**: 提交前由 `pre-commit` 自动运行 ruff 和大文件/私钥检查; PR 前至少执行:

```
ruff check .
ruff format --check .
pytest -q
```

涉及模型调用的测试默认使用 mock; 真实调用需在 PR 中附命令输出或截图。

5. **提交 PR**: 填写改动、原因、验证结果、影响范围和自检清单; 超过 400 行应拆分, 未完成但需要反馈时使用 Draft PR。
6. **审核与合入**: reviewer 检查代码、接口、测试证据和敏感信息; 作者处理评论并 Resolve conversation。满足条件后使用 Squash merge。
7. **归档与复盘**: 合并后删除分支; 里程碑完成立即打 tag, 过程材料当天存入 `process/`。

3. 分支、提交与 PR 规范

3.1 分支命名

格式为 `<type>/<issue-id>-<short-description>`, 全小写、连字符分隔、不使用中文与空格、不超过 50 个字符。

前缀	用途	示例
<code>feat/</code>	新功能	<code>feat/12-memory-extraction</code>
<code>fix/</code>	缺陷修复	<code>fix/34-json-encoding</code>
<code>docs/</code>	文档或 slides	<code>docs/7-milestone1</code>
<code>refactor/</code>	不改变行为的重构	<code>refactor/20-retrieval-api</code>
<code>test/</code>	测试补充	<code>test/22-recall-eval</code>
<code>chore/</code> / <code>exp/</code>	配置维护 / 探索实验	<code>chore/5-ruff-setup</code>

3.2 约定式提交

格式：<type>(<scope>): <描述>，描述用一句话说明做了什么，不加句号。常用类型为 feat、fix、docs、test、refactor、chore、perf、style。

```
feat(memory): 抽取人物与时间字段
fix(eval): 修正无命中时的 Recall@K 计算
docs(milestone1): 补全团队分工与会议机制
```

破坏性变更在 type 或 scope 后加 !，并在正文或 footer 说明 BREAKING CHANGE。

3.3 PR 最低要求

PR 标题沿用约定式提交格式，因为 Squash merge 后标题会成为 main 上的提交信息。描述至少包含：

- 改了什么
- 为什么改 (Closes #编号)
- 怎么验证的 (命令、结果、截图)
- 影响范围 (是否影响他人模块的接口)
- 自检
 - [] 本地跑过 lint 与测试
 - [] 未提交密钥、数据或大文件
 - [] 接口改动已通知消费方

4. 角色与文件边界

成员	主责	主要范围	协作边界
陈易	项目管理与集成架构	里程碑、周会、仓库权限、分支保护、整体架构、集成、部署与演示	不负责核心算法、评测脚本和文档技术章节
鲁瑞特	数据、记忆与检索基础设施	src/memory/、src/index/、Memory 结构、LoCoMo 清洗、索引与检索接口、依赖版本	不负责状态关系判定、prompt、前端和评测
于国庆	动态状态解析	src/reasoning/、Query 分析、RETAIN / UPDATE / SUPERSEDE / CONFLICT、prompt、答案生成	不修改 Memory 结构和索引实现
许子萌	Web 应用与可视化	src/web/、前端、API、证据视图、状态演化可视化和演示素材	不负责 Memory 结构、推理逻辑和评测脚本
张崇洋	评测与文档主笔	src/eval/、experiments/、docs/、slides/、实验记录、图表、Markdown/PDF 定稿	不负责核心算法和前端实现

文件归属规则：目录结构与集成入口由陈易维护；Memory 与检索由鲁瑞特维护；推理与 prompt 由于国庆维护；Web 与 API 由许子萌维护；评测与 docs/ 主文档由张崇洋维护。其他成员通过 PR 供稿，不直接覆盖归属人的文件。

备份配对：记忆/检索 ↔ 于国庆；状态解析/prompt ↔ 鲁瑞特；Web/可视化 ↔ 陈易；评测/文档 ↔ 陈易；计划/仓库 ↔ 张崇洋。发生交接时，在 process/ 记录交接内容、当前状态和后续责任人。

5. 会议、记录与里程碑

- 每周召开一次项目周会，组织者固定为 陈易；会议聚焦进度、阻塞、接口变更和下一周行动项。
- 纪要由成员轮值，当天上传至 process/；每条行动项必须有负责人、截止时间和验收结果。
- 每个里程碑结束时复盘交付、延期、接手和流程问题，并同步 README.md、周计划和本表。

项目周	日期	关键节点	纪要记录者
第 1 周	9/28-10/4	建仓库、定分工、里程碑 1 与 proposal 初稿	鲁瑞特
第 2 周	10/5-10/11	10/9 proposal 展示；10/10 交里程碑 1、2	于国庆
第 3 周	10/12-10/18	记忆构建与检索骨架跑通	许子萌
第 4 周	10/19-10/25	10/22-23 中期展示；10/24 交里程碑 3、4	张崇洋
第 5 周	10/26-11/1	功能冻结，运行对照实验	鲁瑞特
第 6 周	11/2-11/9	最终展示；11/9 交最终报告	于国庆

里程碑主责：团队工作流由 陈易、张崇洋 主责；Proposal 由 陈易 统筹、许子萌 主责展示；需求规格、中期设计测试和最终报告由全员按模块供稿，张崇洋 负责文档收口，陈易 负责集成与交付。

6. 冲突、质量与完成标准

1. 冲突只在个人分支解决：先同步最新 main，解决后重新运行本地检查，再推送。
2. 不合并未解决的冲突，不使用 --no-verify，不提交 API key、.env、数据集或模型输出大文件。
3. 两人同时需要修改同一文件时，以文件归属人为最终维护者；新增、移动或重命名顶层目录必须在周会上提出并由陈易执行。
4. 任务只有在代码或文档完成、验证证据已记录、PR 评论已处理、必要的交接和过程材料已归档后，才算完成。
5. 里程碑交付后立即打 tag，例如：

```
git tag -a v0.1-milestone1 -m "Milestone 1 Team Workflow"
git push origin v0.1-milestone1
```